

Maxon Internship

Realization Document

Fanni Véh
Student Bachelor Applied Computer Science

Table of Contents

1. INTRODUCTION	4
1.1. Background Information	4
1.2. Scope and Delimitation	4
1.2.1. In Scope:	4
1.2.2. Out of Scope:	4
1.3. Structure of the Document	5
2. ANALYSIS	6
2.1. The Existing System	6
2.2. Development Environment Choices	7
2.2.1. Deployment format: snap instead of Docker	7
2.2.2. Database: SQLite instead of PostgreSQL	8
2.2.3. Development environment: local setup instead of the App Build Environment	9
2.2.4. HTTP client: httpx instead of requests	9
3. IMPLEMENTATION PROCESS	10
3.1. Refactoring the Pipeline for the Sandbox	10
3.2. The Snap Packaging Workflow	10
3.3. Interface Integration	10
3.4. Developing the Live Motor Endpoints	11
3.5. Bypassing Sandbox Restrictions for Log Access	12
3.6. Version Parity and Migration	12
4. APPLICATION AND RESULTS	13
4.1. The Final ctrlX Application	13
4.2. Live Motor Interface and API Endpoints	14
4.3. Dynamic Training and Prediction Workflow	15
4.4. Accessible System Logging	16
4.5. Final Version Integration	17
5. TESTING AND VALIDATION	18
5.1. Physical Testing Methodology	18
5.2. Model Accuracy and Resolution	18
5.3. Evaluation Against Success Criteria	19
5.4. Known Limitations	19
6. CONCLUSION	20
6.1. Recap	20
6.2. Project Summary	20
6.3. Evaluation	20
6.4. Look Ahead	20
REFERENCE LIST	22

1. Introduction

The realization document describes the execution and realizations of the internship project at Maxon Motor in Sachseln, Switzerland. The final goal, as defined in the project plan, was to integrate the previously established MIND AI pipeline into a ctrlX app that is deployable via the ctrlX App Store.

1.1. Background Information

Maxon Motor was founded in 1961 as a production site for an electrical engineering company in Sachseln, Switzerland. The first breakthrough came in 1969 with the development of the first DC motor with an ironless rotor, which almost doubled the efficiency in comparison with traditional motors. After this achievement, Maxon became a registered trademark. Since then, Maxon Motor has been mainly focusing on creating electric motors for several different sectors; for example, the very first rover that ever landed on Mars has eleven Maxon motors on board, and they have since been used in several other Mars rovers. However, they are not only focused on the aerospace industry, having several solutions in medical technology and industrial automation. They have a presence in over 40 countries, with 9 production sites, including the USA, Australia, South Korea, and many more (Maxon Group, n.d.).

The main focus of this document will be a fairly recent development titled "Motion Insights and Diagnostics," or "MIND" for short. It is a predictive maintenance tool, where you can track the condition of motors in a drive system. The motor itself acts as the sensor; it reports if a problem arises. With this tool, failures can be detected before they would cause serious issues, and therefore, downtime can be prevented. For instance, using a motor in a breathing machine can help detect failures before they occur, ultimately saving lives.

1.2. Scope and Delimitation

To ensure the project remained focused and achievable within the 17-week timeframe, clear boundaries were established regarding what functionalities would be integrated and what would be excluded.

1.2.1. In Scope:

- Refactoring AI pipeline for sandbox execution
- Integration into ctrlX App architecture
- Data Layer¹ communication
- Defined REST API
- Basic UI aligned with ctrlX guidelines
- Snap packaging² & metadata
- Documentation
 - Realization document
 - Implementation Guide

1.2.2. Out of Scope:

- Training new AI models
- Cloud-based AI services
- Large-scale distributed systems
- Advanced UI redesign beyond ctrlX standards
- Continuous deployment automation
- Performance optimization beyond required limits
- Hardware redesign
- Cybersecurity certification beyond store requirements

¹ This includes reading motor axis values such as current draw and timestamps through the ctrlX Data Layer API, as well as writing inference results back to the system through defined endpoints.

² This involves building the application into a confined snap format compatible with ctrlX OS using snapcraft, including all required configuration files and metadata necessary for potential App Store submission.

1.3. Structure of the Document

This document describes the steps taken during the internship, covering the tools and technologies selected, the technical challenges faced, and the achieved results.

This first chapter introduced the context of the project. The remaining document will be structured as follows: it starts with Chapter 2, which covers the existing system and the technical choices made. Chapter 3 details the implementation process, including refactoring and packaging the application. Chapter 4 provides an overview of the application & results obtained, while Chapter 5 outlines the testing & validation phases. The final chapter presents a conclusion and a look ahead for the future. At the end of the document, references and attachments can be found.

2. Analysis

Before any implementation could begin, it was necessary to spend time understanding both the system that already existed and the constraints of the environment it would need to run in. This chapter covers the outcome of that analysis: what the MIND system is, how it is structured, and which technical choices were made as a result, including the reasoning behind them.

2.1. The Existing System

Maxon MIND (Motion Insights and Diagnostics) is a predictive maintenance platform for electric motors. As previously mentioned, the motor itself serves as the sensor: by continuously recording data signals such as the current, velocity, torque, and position during operation, it is possible to detect early signs of wear or degradation before they lead to an actual failure. This is particularly valuable in applications where unexpected downtime has serious consequences, for example, in medical devices.

The application is built in Python using FastAPI as the web framework (FastAPI, n.d.), SQLAlchemy as the database ORM, and Jinja2 combined with HTMX for the frontend. The choice of FastAPI is relevant because certain operations, most notably model training, can take several seconds. An asynchronous framework ensures the server remains responsive during those operations rather than blocking all other requests.

The data is organized in a hierarchy that reflects the physical structure of the system. At the top level there are motors, each representing a physical drive unit identified by its serial number and part number. A motor can have multiple applications, where each application represents a specific configuration or use case; for example, running the same motor at different load settings or target positions would result in separate classifications, since the sensor signals would differ enough to require separately trained models. Each application stores many cycles, which represent individual test sequences: one complete movement from the home position to a target position and back. Each cycle consists of hundreds of timestamped measuring points, sampled at approximately five-millisecond intervals across nine sensor channels.

On top of this data, the system manages training runs and prediction runs. A training run takes a selection of classified cycles (labelled either good or bad) and produces a machine learning model. A prediction run then takes that model and scores new, unclassified cycles against it to determine the current health of the motor.

The machine learning pipeline is the core of what makes MIND useful, and it is worth explaining in some detail how it works, since the decisions made in its design have a direct impact on how reliable the predictions are.

Every time a motor completes a test cycle, the result is a sequence of sensor readings over time, essentially a table where each row is a point in time and each column is a measurement channel such as current, velocity, or position. A typical cycle produces several hundred of these rows. The challenge is that machine learning models require every input to be the same size and shape, which raw time-series data is not: cycles can have slightly different lengths depending on how the motor behaved, and there is no straightforward way to compare them directly.

The solution is feature extraction. Instead of feeding the raw signal into the model, a large set of descriptive statistics is computed for each cycle, such as the average current over the whole cycle, how much it fluctuated, how many peaks it had, what the dominant frequencies in the signal were, and dozens of others. In total, over a hundred such features are computed per cycle. After this step, every cycle is represented as a single row of numbers of fixed length, regardless of how long the original signal was. This structure is what is transferred to the model.

However, having over a hundred features is itself a problem. Many of them will carry overlapping or redundant information; for example, if the maximum current goes up, the average current likely does too, so both features are not telling the model anything meaningfully different. Working with too many correlated features makes models less reliable and harder to train, a well-known issue in machine learning referred to as the “curse of dimensionality.” For this reason, dimensionality reduction is applied before training: the goal is to go from over a hundred features down to a much smaller set that still captures the most useful information.

The pipeline supports several methods for doing this. PCA (Principal Component Analysis) and SVD (Singular Value Decomposition) are mathematical techniques that find new combined features that capture as much of the variation in the data as possible. The top components are kept, and the rest are discarded. A simpler alternative is correlation-based selection, which simply removes features that are too similar to other features already in the set, keeping the originals rather than creating new combined ones.

The most powerful method, however, is Fisher score selection. Rather than looking at how much a feature varies in general, a Fisher score measures how well a feature separates the two classes (good cycles versus bad cycles). A feature scores highly if the average value for good cycles and the average value for bad cycles are far apart, while the values within each group remain consistent. In other words, a high-scoring feature is one where healthy and faulty motors genuinely behave differently. The algorithm does not simply pick the highest-scoring features individually, though; instead, it finds the best combination of features, ensuring that the selected features are not only discriminative but also sufficiently different from each other. Including two features that essentially measure the same thing would waste one of the available slots. Importantly, the average Fisher score of the final selection is also used as a data quality check: if it falls below a set threshold, the system raises a warning, because a low score suggests that the training data does not contain a clear enough difference between the “good” and “bad” cycles to train a reliable model. In the MIND pipeline, Fisher score selection is the method applied for dimensionality reduction before model training.

Once the feature set has been reduced, the model is trained using HDBSCAN, a clustering algorithm (McInnes, Healy, & Astels, 2017). The idea is that when healthy cycles are plotted in the reduced feature space, they tend to group together, while faulty cycles either form a separate group or appear as outliers far from the healthy cluster. The algorithm learns where these groups are during training, and at prediction time it places a new cycle in the same space and determines which group it belongs to.

Finally, the trained model is stored in ONNX format (Open Neural Network Exchange) (ONNX, n.d.). ONNX is a standardized, platform-independent format for machine learning models. This is significant because the ctrlX CORE runs on an ARM64 processor with limited software, and it would not be practical to install the full Python machine learning library just to run predictions. With ONNX, the model can be loaded and used on any compatible platform without the training environment needing to be present.

The first few weeks of the internship were dedicated to understanding all of this in practice: reading through the code, exploring the file structure, and testing the application manually with guidance from colleagues.

2.2. Development Environment Choices

With a clear understanding of the existing system in place, several technical decisions had to be made before implementation could begin. Most of these were shaped by the constraints of the ctrlX CORE platform, which imposes specific requirements on how applications are packaged and run.

2.2.1. Deployment format: snap instead of Docker

The original development environment implemented Docker containers. However, ctrlX CORE only supported Snap packages as a deployment format for third-party applications; therefore, that was the applied framework. The main difficulty with using Snap was that building a snap for a Python-heavy project

with many dependencies is an extremely slow process by default; it was imperative to analyze how to work with snap packaging efficiently.

Out of the optimizations found, the most impactful one was keeping the project files inside WSL (Windows Subsystem for Linux) and running Snapcraft from there. With WSL, Snapcraft caches files it has already processed (Canonical, n.d.), allowing subsequent builds to only repackage the files that actually changed. This reduced build times from over an hour to a matter of minutes for incremental changes.

For detailed technical information, please refer to Appendix A.

2.2.2. Database: SQLite instead of PostgreSQL

The existing MIND application used PostgreSQL. For a standard server deployment this is a reasonable choice, but for an embedded controller it introduces unnecessary complexity. PostgreSQL requires a separate server process, adds to the snap package size, and is significantly heavier in terms of resource usage. Since the expected scale of data (cycles from one or a small number of motors) does not come close to requiring the features PostgreSQL offers, SQLite was the better fit. It is fully self-contained, requires no separate process, and integrates cleanly with the existing SQLAlchemy ORM layer with minimal changes (SQLite, n.d.).

To properly justify the decision behind choosing SQLite, a Weighted Ranking Matrix was utilized. Five criteria were identified and weighted on a scale of 1 to 5 (where 5 is highest importance). Each database was then scored from 1 to 5 on how well it fulfilled each criterion.

Table 1: Weighted Ranking Matrix - Database Selection

Evaluation Criteria	Weight	SQLite (Score)	SQLite (Weighted)	PostgreSQL (Score)	PostgreSQL (Weighted)
Resource Efficiency (RAM/CPU)	5	5	25	2	10
Snap Packaging Compatibility	5	5	25	1	5
Zero-Configuration Maintenance	4	5	20	2	8
Multi-User Scalability	2	2	4	5	10
Advanced Query Features	2	3	6	5	10
Total Score			80		43

Justification of Criteria and Results:

- Resource Efficiency & Packaging (Weights 5): The ctrlX controller operates on an ARM64 architecture with limited hardware resources. PostgreSQL requires a separate, continuously

running daemon process, which significantly inflates the snap package size and consumes background memory. SQLite is entirely self-contained within the application process, scoring perfectly here.

- Zero-Configuration (Weight 4): Since the goal is a distributable app via the ctrlX App Store, clients should not be required to configure database users, roles, or networking. SQLite handles data locally in a simple file, eliminating setup overhead.
- Scalability & Advanced Features (Weights 2): While PostgreSQL heavily outperforms SQLite in high-concurrency environments and advanced relational queries, the MIND app's local instance only needs to store cycle data for one (or a very small number of) connected motors. The advanced capabilities of PostgreSQL are effectively wasted in this context.

Given the final weighted score of 80 to 43, SQLite was the clear, objective choice. It integrates cleanly with the existing SQLAlchemy ORM layer, requiring minimal refactoring of the original pipeline while adhering perfectly to the constraints of the hardware.

2.2.3. Development environment: local setup instead of the App Build Environment

The ctrlX Works software includes a virtual environment called the 'App Build Environment' that provides a command-line interface (CLI) for accessing example code and building snaps. This environment, due to its simplicity, was used in the early stages of the project to become familiar with the ctrlX App SDK (Automation Community, 2020). The limitations of this solution, however, are that through the CLI, there is no visibility into the broader file structure, making it impractical at scale. To solve the scalability issue, a proper integrated development environment (IDE) would be needed. Fortunately, the previously mentioned WSL environment was accessible locally in Visual Studio Code via SSH. This setup allowed for a full development workflow with proper file navigation, code completion, and version control without any of the limitations of the CLI-only environment. From that point on, all further development was done locally in VS Code.

2.2.4. HTTP client: httpx instead of requests

For communicating with the ctrlX Data Layer API (the internal interface that provides applications access to motor axis commands and sensor values), an HTTP client library is needed. The standard choice in Python is the requests library, which is reliable and familiar to most developers. However, when the application was first packaged as a snap and tested on the ctrlX controller, requests produced SSL errors when attempting to connect to the data layer. The issue arises from requests that rely on the operating system's certificate store to verify HTTPS connections, and within the snap sandbox, that certificate store isn't accessible in the same way as in a standard environment.

The httpx library was used as a replacement. It handles SSL verification independently from the operating system, making it fully compatible with the snap environment (Encode, n.d.). The practical difference in the code is minimal, as the two libraries share almost identical syntax, but the underlying behaviour is different enough to matter in this context. Since the ctrlX controller uses a self-signed certificate by default, SSL verification is disabled in the Data Layer client; however, httpx is still necessary because requests fail at the connection level before that setting even takes effect. This failure is an excellent example of how constraints that are invisible during local development only surface once the application is actually running on the target hardware. It should be noted that in the current implementation, SSL certificate verification is disabled to allow communication within the ctrlX local network, where self-signed certificates are in use. This is acceptable in a closed, device-local environment. However, if the application were ever to communicate over a public or external network, proper certificate validation would need to be re-enabled to avoid security vulnerabilities.

3. Implementation Process

This chapter describes the actual steps taken to build the application, the challenges encountered along the way, and how they were resolved. The development did not follow a perfectly linear path; problems surfaced at each stage that required investigation and iteration before the progress could continue.

3.1. Refactoring the Pipeline for the Sandbox

The existing MIND codebase was developed as a standard Python web service, built to run locally with Docker and a PostgreSQL database. Before any snap packaging could begin, the application needed to be stripped and adapted to run in a fundamentally different environment.

The first step was removing the Docker configuration files and replacing any dependencies that assumed the presence of external services. The database connection was updated from PostgreSQL to SQLite, with the database file stored in the snap's designated writable data directory. Every file path used in the application had to be verified and corrected to point to locations within the snap filesystem rather than hard-coded local development paths.

The more technically involved change was the HTTP client used to communicate with the ctrlX Data Layer. The original code used the requests library, which failed with SSL certificate verification errors when running inside the snap sandbox. As the snap environment restricts access to the system certificate store that the request relies on, the library simply could not establish a connection. Switching to httpx, which handles certificate verification independently from the operating system, resolved the issue entirely, as described in the previous chapter.

3.2. The Snap Packaging Workflow

Once the codebase was adapted, the next challenge was getting the snap to build reliably and in a reasonable amount of time. The initial builds were extremely slow, in many cases taking well over an hour, because Snapcraft was downloading and installing all Python dependencies from scratch on every build. For a project with nearly a hundred packages, this made iterative development impractical.

The solution came from understanding how Snapcraft's caching mechanism works (Canonical, n.d.). Snapcraft caches which files it has already processed, but this cache only persists if the build runs from the same filesystem location. By copying the entire project structure into WSL and running Snapcraft from there, the cache was preserved between builds. After the first full build, only files that had actually changed needed to be reprocessed, reducing subsequent build times from over an hour to a matter of minutes. This became the standard workflow for the rest of the internship.

An additional complexity was that the ctrlX CORE runs on an ARM64 processor, while development was done on a standard AMD64 machine. The snap therefore needed to be cross-compiled, meaning it was built on one architecture and targeted at another. This setting was configured in snapcraft.yaml manifest and handled automatically by snapcraft. Cross-compilation added to the initial build time and meant that certain packages had to be sourced as precompiled wheels rather than built from source.

3.3. Interface Integration

Integrating the web interface went through several distinct stages. The starting point was the ctrlX App SDK, which includes a basic web server example (a minimal HTTP server serving a static page) intended as a scaffold for developers building their first ctrlX application (Automation Community, 2020). This was used to verify that a web application could be packaged and served through the ctrlX's built-in Nginx reverse proxy before attempting to integrate anything more complex.

From there, a static HTML page was built to confirm that the routing, file serving, and authentication flow all worked correctly on the controller. Only once this was stable was the full MIND frontend introduced.

This stage brought a new category of problem: version mismatches between Python packages installed in the snap and those expected by the application. The most visible symptom was the application hanging on an initialization screen after login without producing a clear error. Tracing this through the logs revealed that certain template rendering failures were being swallowed silently, and the root cause was a dependency version conflict that only surfaced in the snap environment and not during local development.

A subtler but equally impactful issue was the Nginx URL prefix. The ctrlX reverse proxy routes all application traffic through a /mind path prefix, which it strips before forwarding requests to the application. This worked correctly for page loads, but all JavaScript fetch calls in the frontend were using root-relative URLs such as /api/live/status. Inside the snap, these requests were going to the wrong path and failing silently. The fix was to detect the prefix at runtime and prepend it to all API calls:

```
const BASE = window.location.pathname.startsWith('/mind') ? '/mind' : '';
```

This single change resolved all the broken API calls in the snap environment without affecting local development, where the prefix is not present.

3.4. Developing the Live Motor Endpoints

The live capture feature required building a new set of REST API endpoints from scratch, since nothing in the existing MIND codebase handled direct communication with a running motor. Five endpoints were created under the /api/live prefix: one to test the connection, one to start a capture session, one to stop it and save the data, one to return the current status, and one to discard a session without saving.

The start endpoint powers on the motor axis and launches three background threads: one streaming sensor data via SSE subscription at a target rate of 200 Hz, falling back to 100 ms polling if unsupported, one reading slower-changing values such as temperature once per second, and one executing the motion sequence, commanding the motor to move from the home position to a target position and back, repeatedly, until the session is stopped. Each completed movement is buffered in memory as a cycle.

Several problems surfaced during testing. The first was that the stop endpoint was returning a 422 validation error. The endpoint expected the application ID as a URL query parameter, but the frontend was sending it in the request body, a mismatch that is easy to miss but causes the request to be rejected outright. Correcting the frontend call resolved this immediately.

A more significant issue was that no cycles were being saved to the database despite the motor clearly completing movements. The cause was a minimum sample threshold: the code required at least two hundred sensor samples per cycle before it would be considered valid. At the rate HTTP requests to the data layer could be made (roughly one per hundred milliseconds), a cycle would only accumulate around twenty samples during a two-second movement, never coming close to the threshold. The value was corrected to a formula based on the actual movement duration:

```
min_samples = max(5, int(self._time_on_s / 0.1 / 2))
```

The formula derives from the actual sampling rate of the HTTP polling fallback, which collects one sensor reading every 100 ms (PDO_PERIOD_S = 0.1). For a movement of duration time_on_s seconds, the theoretical maximum number of samples is time_on_s / 0.1. Dividing by 2 sets the acceptance threshold at 50% of that maximum, providing a realistic margin for HTTP request latency and processing overhead. The outer max(5, ...) ensures a hard floor of five samples, preventing zero-sample cycles from being stored regardless of how short the configured motion time is. This ensured the threshold was always reachable in practice.

The stopping sequence also required careful handling of the motor's internal state machine. Sending a power-off command while the motor was mid-movement caused the ctrlX to enter an error state, rejecting further commands. The solution was to let the current movement complete naturally before powering off, rather than interrupting it. This was achieved by removing the early exit condition from the position-wait loop, so the background thread always finishes its current cycle cleanly before responding to a stop signal.

3.5. Bypassing Sandbox Restrictions for Log Access

MIND has a thorough logging system that provides detailed logs for each training and prediction run. During local development, these logs are accessible as files saved in a defined directory. Inside the snap, however, the logs are written to a path within the snap's confined filesystem that is not directly browsable through the ctrlX web interface. When issues occurred on the controller, there was no straightforward way to see what had happened.

The solution was to expose the log files through a dedicated URL endpoint. The proposed endpoint reads the contents of the log directory and returns a rendered HTML file. This file would then list all available log files, each with a link to view or download the content directly in the browser. This turned the logs from being an inaccessible internal detail into a practical debugging tool accessible from anywhere on the network, which proved useful throughout the rest of the project.

3.6. Version Parity and Migration

MIND was actively developed by the broader team throughout the internship, and by the later stages, the main repository had moved to version 1.2.3, which included significant changes to the pipeline and data model. It was important that the ctrlX application could be kept up-to-date with these changes rather than diverging permanently into its own branch.

Merging the new version was not straightforward. The changes introduced in 1.2.3 touched many of the same files where the live capture endpoints and interface modifications had been made, which led to conflicts that had to be resolved carefully. The approach taken was to add the new features incrementally rather than attempting a single large merge, verifying at each step that the application still functioned correctly before moving on to the next change. By the end of this process, the updated version was running locally with all of the previously developed features intact, ready for snap packaging and deployment to the controller.

4. Application and Results

This chapter presents the finished application in its final state, running on the ctrlX CORE controller. It focuses on what the completed system looks like and what a user can do with it.

4.1. The Final ctrlX Application

The final application is accessible through the ctrlX web interface alongside any additional installed applications. Once the application opens, the MINDscape dashboard is displayed.

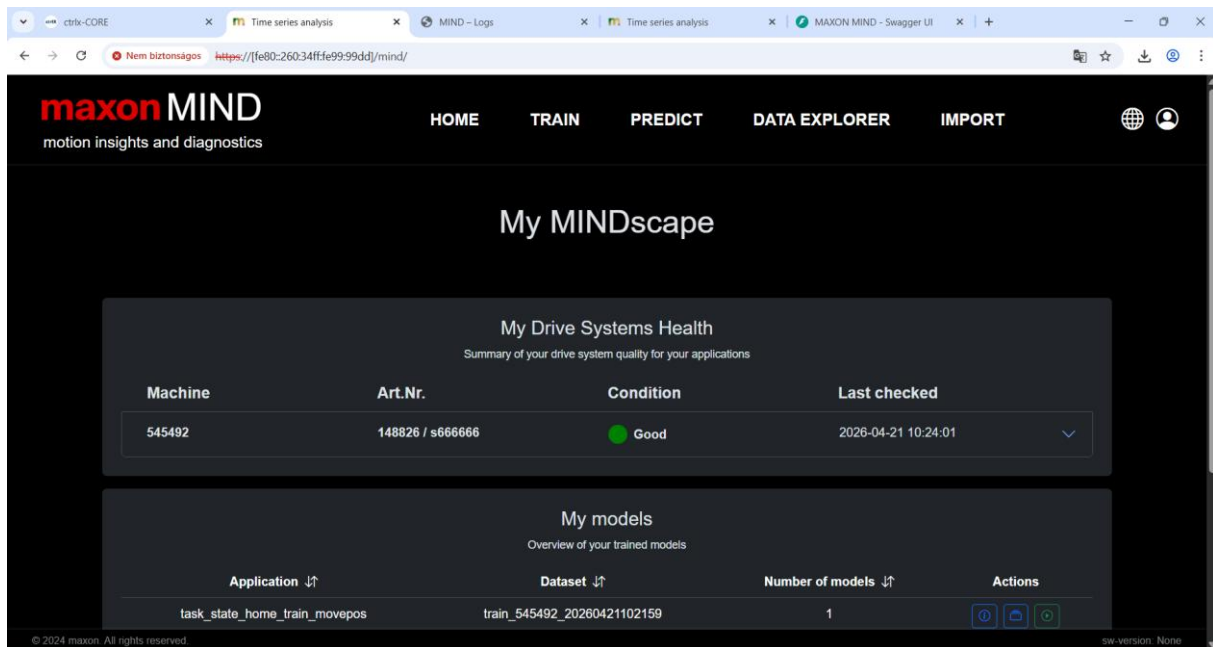


Figure 1: MINDscape Dashboard

As shown in Figure 1, the dashboard provides an immediate overview of the monitored motors and their current health status, displayed as a table showing the machine identifier, article number, condition, and the date of the last prediction. A color indicator gives a quick visual signal of the current state: green for good, orange for used or worn, and red for critical. Below the health overview, the user's trained AI models are listed with their associated application and dataset.

Navigation across the application is handled through a top menu bar with five sections. Home returns to the dashboard. Train opens the workflow for creating a new AI model from cycle data. Predict allows the user to run a health prediction using a trained model. Data Explorer provides a detailed view of all stored cycle data, model runs, and historical predictions. Import allows new cycle data to be uploaded as a ZIP file for applications where live capture is not used.

The application supports both English and German, switchable at any time through the language selector in the navigation bar, making it accessible to users across different regions.

4.2. Live Motor Interface and API Endpoints

One of the central deliverables of this project is a set of REST API endpoints that allow direct communication between the application and a physically connected motor. These endpoints are accessible both through the application's user interface and externally through any HTTP client, making them suitable for integration into automated workflows or testing setups, as detailed in Figure 2.

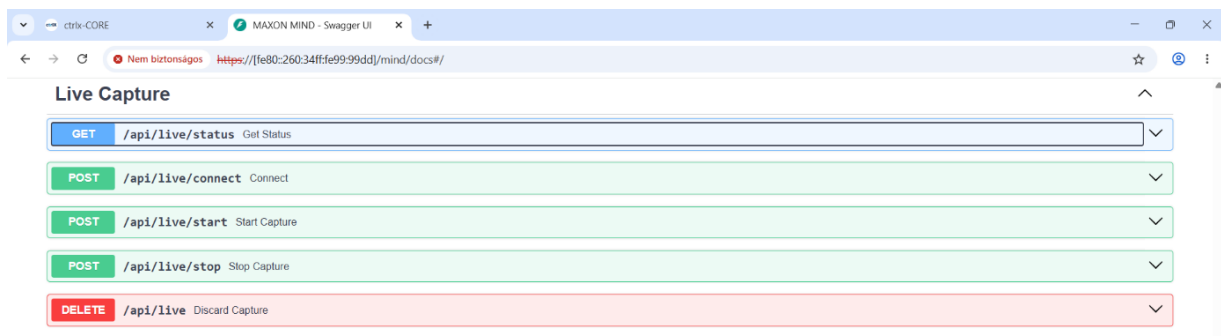


Figure 2: API Endpoints

The Swagger UI, accessible through the application, documents all five endpoints clearly. Their functions are as follows:

GET /api/live/status returns the current state of the capture session. If a session is active, it reports whether the motor is running, how many cycles have been buffered in the current session, and a breakdown of those cycles by classification. If no session is active, it returns an idle status.

POST /api/live/connect tests the connection to the ctrlX system using the provided credentials without starting a capture session. This allows a user to verify that the IP address, username, and password are correct before committing to a full run.

POST /api/live/start authenticates with the ctrlX, powers on the motor axis, and begins the automated cycle sequence. The request body specifies the connection details alongside a set of motion parameters (target position, velocity, acceleration, movement duration, and pause time) as well as the classification to assign to all cycles recorded in this session (good, bad, or unknown). An example request body is shown on the right side of this page.

POST /api/live/stop halts the cycle sequence, stops the motor, and saves all completed cycles from the session to the database, where they become immediately available for training or prediction.

DELETE /api/live stops the session and discards all buffered data without saving anything. This is useful when a test run produced unusable results and should not be added to the dataset.

```
{
  "application_id": 1,
  "ctrlx_ip": "192.168.1.1",
  "username": "boschrexroth",
  "password": "...",
  "axis_name": "Axis_1",
  "classification": "good",
  "target_pos": 90,
  "velocity": 3600,
  "acceleration": 3600,
  "time_on_s": 2,
  "time_pause_s": 0.5
}
```

To further improve usability and model reliability, additional interface elements were introduced. Rather than running cycles individually, the user can now specify the desired number of good and bad cycles upfront. The system then automatically runs the full sequence and shuts off the motor once complete. This ensures a balanced training dataset without requiring continuous manual attention from the user.

The screenshot displays a web interface titled "Connection to ctrlX motor" with the subtitle "Here you can connect and run cycles on the motor". It is divided into two main sections: "Training Mode" and "Individual cycles".

Training Mode: This section contains two input fields: "Cycles per phase" set to 0 and "Position P-gain factor for bad cycles (e.g. 0.5 = half gain)" set to 1.0. There is an "Extended settings" button with a dropdown arrow.

Individual cycles: This section features a "Classification" dropdown menu currently set to "unknown" and another "Extended settings" button with a dropdown arrow.

Summary and Action: On the right side, there is a "Status" box showing "Standby" and a "Cycles this run" box showing "0". Below these are three input fields for "Good cycles", "Bad cycles", and "Unknown cycles", all currently set to 0. At the bottom right, there is a red "Start" button and an "Application ID" field containing the number 1.

Figure 3: Updated Interface with Training Mode

4.3. Dynamic Training and Prediction Workflow

The live motor interface enables a complete end-to-end workflow that does not rely on externally provided datasets. A user can generate all the data they need directly from their connected motor and use it to train and run predictions within the same application.

The process begins by running a set of cycles classified as good. These cycles are recorded while the motor is operating under normal, healthy conditions. These are started through the `/api/live/start` endpoint or the corresponding interface, with the classification field set to good, and run until a sufficient number of cycles have been collected (at least 5 good and 5 bad for training). The same process is then repeated with the classification set to bad, this time recording cycles under known fault conditions, such as an added load or a simulated mechanical issue. Both sets of cycles are saved to the database under the chosen application.

Generating reliable bad cycles without manual intervention was one of the final challenges of the internship. Initially, bad cycles were created by manually applying friction to the motor, which was not a scalable or reproducible solution. Later attempts used the torque offset and velocity factor as alternatives, but these produced models with low accuracy. The final solution was found by directly connecting to the motor to access and modify the position proportional gain (P-gain) through code, bypassing the need for the ctrlX Motion or PLC applications entirely. After extensive tuning of the P-gain value alongside other motor parameters, the application can now generate a reliable predictive model fully automatically, without any manual intervention on the local machine (more details in chapter 5.4. Known Limitations).

With good and bad cycles stored, the user navigates to the Train section of the application. The available cycles are displayed with a preview, and the user confirms the selection before starting the training run. The pipeline processes the data, extracts features, applies dimensionality reduction, and trains the model as described in Chapter 2. Once complete, the new model appears in the models list on the dashboard.

Predictions can then be run on unknown cycles, which are collected from the motor under any operation without a known classification. These are recorded in the same way through the live interface, this time with the classification set to unknown. In the Predict section, the user selects the trained model and the unknown

cycles, and the application scores each cycle and produces a health assessment. The results are visible in the data explorer and can be downloaded as a report.

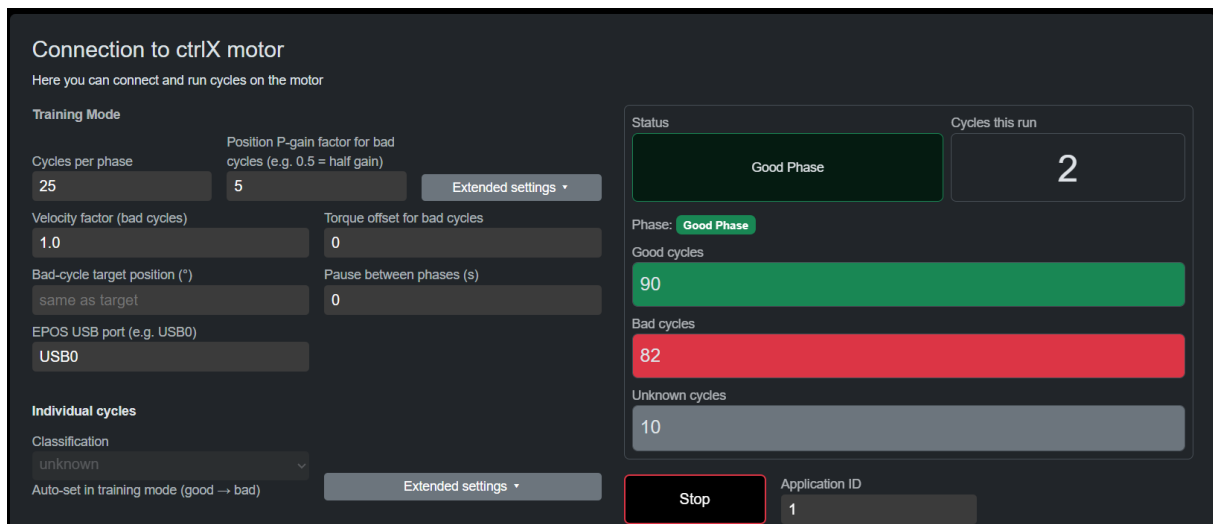


Figure 4: Live Motor Interface

4.4. Accessible System Logging

The application includes a log viewer interface that exposes the internal logs of the MIND system through the browser, without requiring any command-line access to the controller (Figure 5).

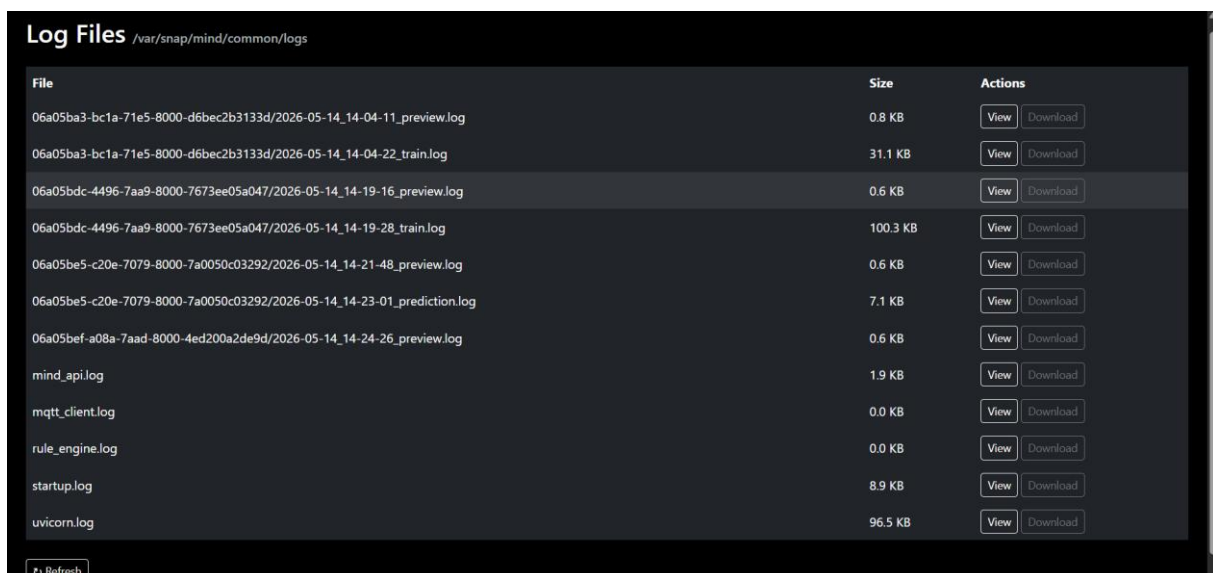


Figure 5: Log Files Interface

The log viewer lists all available log files by name, including separate logs for training runs, prediction runs, and system events. Each file displays its size and provides two actions: **View**, which opens the log content directly in the browser, and **Download**, which saves the file locally. The files are named with the run identifier and a timestamp, making it straightforward to find the log corresponding to a specific operation. This interface is particularly useful for diagnosing issues on the controller, where the underlying filesystem is not directly accessible to the end user. An administrator can inspect the full output of a training or prediction run without having to leave the web interface.

4.5. Final Version Integration

In the later stages of the project, the application was updated to incorporate version 1.2.3 of MIND, which was the current version being developed by the other team members, as shown in the bottom right corner of Figure 5.

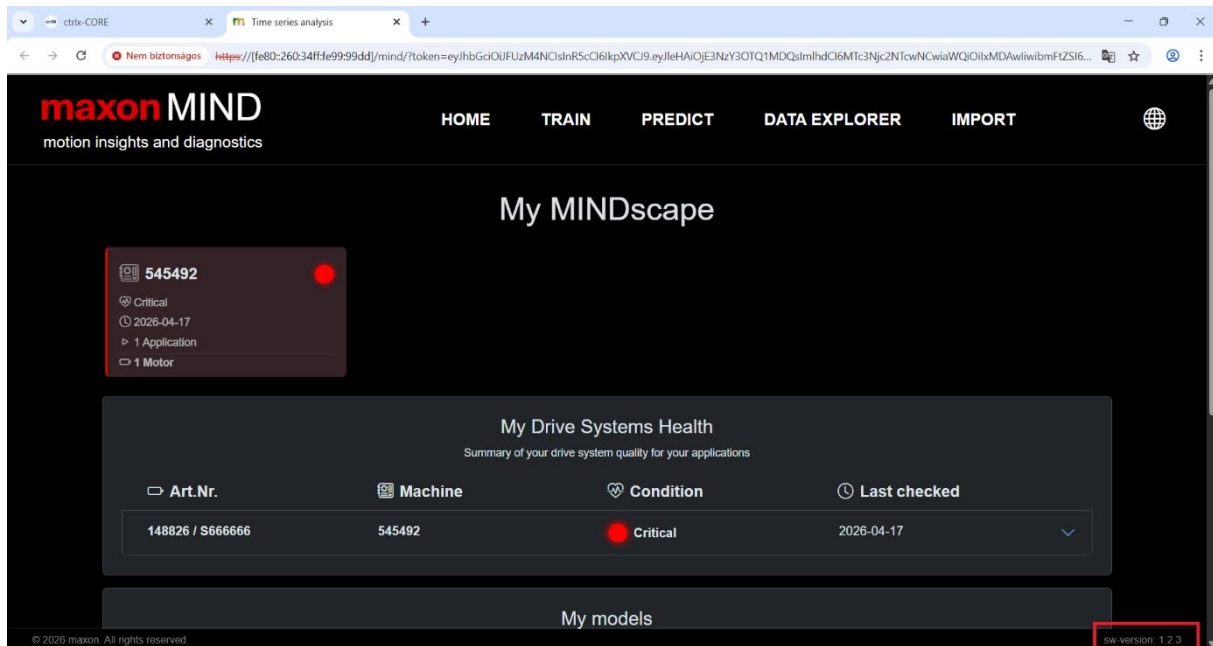


Figure 6: MINDscape version 1.2.3

The updated version includes improvements to the dashboard layout, additional data fields in the motor overview, and changes to the internal pipeline that improve model training stability. All features developed during this internship were successfully migrated into the new version. The application running on the ctrlX controller is therefore fully up to date with the main development branch, and future updates to MIND can be incorporated using the same migration process.

5. Testing and Validation

This chapter evaluates the finished application against the goals defined at the start of the project. It covers how the physical testing was conducted, how a significant accuracy discrepancy was identified and resolved, and where known limitations remain.

5.1. Physical Testing Methodology

Testing the live capture feature required generating reliable sets of both good and bad cycles directly from the connected motor. A good cycle is straightforward to produce: the motor simply runs its full movement sequence under normal conditions without interference. Generating bad cycles is less obvious when only a single motor is available, as a realistic fault scenario would typically involve a second motor applying a controlled resistive load.

In the absence of such a setup, mechanical resistance was simulated manually. During a capture session classified as bad, physical pressure was applied to the motor shaft by hand while it was in motion, increasing the friction and forcing the motor to draw more current to maintain its commanded velocity and position. This directly affects the sensor signals in a way that is meaningfully different from a healthy cycle. The result is a dataset where the good and bad cycles are distinguishable by the machine learning pipeline, even though the fault was simulated rather than mechanical in origin.

This approach has a clear limitation: it is difficult to apply the same amount of resistance consistently across multiple cycles, which introduces variability in the bad cycle data. Therefore, the focus shifted toward creating distinguishable cycle clusters without manual intervention. Several approaches were explored: adjusting the velocity offset to alter the motor speed during bad cycles, using a torque offset to increase load, and varying movement duration between good and bad cycles. None of these produced sufficiently distinct clusters to train a reliable model. The final solution was introducing P-gain variance, which successfully created distinguishable good and bad cycles. However, as described in section 5.4, this solution currently only works in the local development environment rather than from the deployed controller application.

5.2. Model Accuracy and Resolution

During testing on the ctrlX controller, the prediction results were noticeably worse than those observed when running the same models locally. Cycles that were correctly classified in the local environment were being misclassified on the controller at a significantly higher rate, despite the underlying data being identical.

The root cause was identified in the model selection logic within the prediction pipeline. When multiple trained models are available for an application, the system is intended to select the best-performing one (determined by its stored accuracy metrics) to use for the prediction. A line of code responsible for this ranking had been accidentally removed during the migration to version 1.2.3, causing the pipeline to default to the first model in the list rather than the best one. Depending on the order in which models had been created, this often resulted in an older, lower-quality model being used for predictions.

Once the correct selection logic was restored, the prediction results on the ctrlX matched those produced locally. This confirmed that the accuracy issue was not related to the controller's hardware, the ONNX inference engine, or any difference in the snap environment, but it was entirely a consequence of the model selection bug, and its resolution validated that the pipeline itself was functioning correctly across both environments. Prior to identifying this issue, prediction accuracy on the ctrlX was unreliable and inconsistent with locally produced results. Once the correct model selection logic was restored (ensuring the best-performing model is selected rather than always defaulting to the first) the results aligned fully with local testing.

5.3. Evaluation Against Success Criteria

The original project scope defined a set of deliverables against which the final result can be evaluated. The table below maps each criterion to its outcome:

Table 2: Evaluation against Predefined Criteria

Criterion	Outcome
Refactoring AI pipeline for sandbox execution	Met — the pipeline runs fully within the snap environment
Integration into ctrlX App architecture	Met — the application is installable and accessible through the ctrlX web interface
Data Layer communication	Met — the live capture system reads sensor data and sends motor commands through the Data Layer API
Defined REST API	Met — five /api/live endpoints are implemented and documented via Swagger
Basic UI aligned with ctrlX guidelines	Met — the interface is accessible through the ctrlX and supports both English and German
Snap packaging and metadata	Met — the application packages successfully and installs on the controller
Functional testing	Partially met — core functionality has been tested and validated; few small known bugs remain (see section 5.4)
Documentation	Met — internal code documentation and implementation guide completed; this document finalised

Seven of the eight defined criteria are fully met. The last partial item is functional testing, which is complete for all major features except the issues described below.

5.4. Known Limitations

5.4.1. P-gain solution

The automated bad-cycle generation via P-gain manipulation relies on a straight connection to the motor using a Windows-specific application called Epos Studio. This works correctly during local development on a Windows machine, but the library is not available on the ctrlX CORE's ARM64 Linux environment. As a result, the `delta_p` feature is silently skipped when the application is deployed as a snap: bad cycles are still run, but the P-gain is not modified. Users relying on this feature must currently operate the live capture from a Windows machine connected directly to the motor via USB rather than from the ctrlX web interface.

5.4.2. Small dataset sensitivity

The HDBSCAN clustering algorithm requires a minimum of five cycles per class to form two distinct clusters. With borderline datasets (e.g., exactly five cycles per class with overlapping signals), the clustering may fail and no model is created, even though the data technically meets the minimum count threshold.

6. Conclusion

6.1. Recap

The goal of this internship was to take an existing AI pipeline for motor health monitoring and integrate it into a deployable ctrlX application so that it could be installed on the ctrlX CORE controller, communicate with a connected motor, and run the full training and prediction workflow in a sandboxed environment. Starting from a codebase built for local development with Docker and PostgreSQL, the application was refactored, packaged as a snap, and extended with a live data collection interface that did not previously exist.

6.2. Project Summary

The project demonstrates that a Python-based machine learning application of this complexity can be successfully deployed as a snap on an embedded industrial controller, provided the right adaptations are made. The most significant of these were the switch to a self-contained database, the replacement of the system-dependent HTTP client, and the careful management of the snap build process. None of these were obvious at the outset, and each required investigation before a solution was found. The live motor interface adds a capability that makes the application genuinely self-sufficient; a user can connect their own motor, collect training data, build a model, and run predictions entirely within the ctrlX environment without relying on externally provided datasets. The physical testing methodology, while limited by the available hardware, was sufficient to validate that the pipeline produces meaningful results.

6.3. Evaluation

The majority of the originally defined deliverables have been completed. The application runs within ctrlX sandbox constraints, interfaces with the Data Layer, exposes REST API endpoints, includes a UI aligned with ctrlX guidelines, and has been snap packaged. The extended scope (live motor training and prediction) was also successfully implemented. It should be acknowledged that several colleagues provided essential support in navigating technical challenges specific to the Maxon hardware and software environment, which contributed meaningfully to the outcomes achieved.

6.4. Look Ahead

Several natural next steps follow from the current state of the project. The application would benefit from continued version parity with the main MIND development branch, using the migration process established during the move to version 1.2.3. In the longer term, the application is a strong candidate for submission to the ctrlX App Store, which would make it available to other ctrlX users beyond the Maxon development team. The work done here provides a foundation that the broader team can build on as MIND continues to develop.

7. GenAI Policy

During the internship, Artificial Intelligence tools were frequently used for the following purposes: code analysis, idea generation, code generation, reinforcement, feedback, etc. The tools used include but are not limited to: Claude (including Claude Code), Gemini, ChatGPT and Copilot.

During the creation of this document AI tools were used to ensure grammatical and factual correctness, as well as understandability of the content.

REFERENCE LIST

- Canonical. (n.d.). *Snapcraft documentation*. <https://snapcraft.io/docs>
- Encode. (n.d.). *HTTPX - A next-generation HTTP client for Python*. <https://www.python-httpx.org/>
- FastAPI. (n.d.). *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- Maxon Group (n.d.) The entire world of drive technology | maxon group. <https://www.maxongroup.com/en>
- McInnes, L., Healy, J., & Astels, S. (2017). hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software*, 2(11), 205. <https://joss.theoj.org/papers/10.21105/joss.00205>
- ONNX. (n.d.). *Open Neural Network Exchange*. <https://onnx.ai/>
- SDK for ctrlX AUTOMATION. (2020, September 28). *Automation Community*. <https://community.boschrexroth.com/ctrlx-os-store-apps-oc2pqgwn/post/sdk-for-ctrlx-automation-xDk5sDAU4HTxpO7>
- SQLite. (n.d.). *About SQLite*. <https://www.sqlite.org/about.html>

ATTACHMENTS

APPENDIX A – TECHNOLOGY GLOSSARY

- **FastAPI:** A modern, fast web framework for building APIs (Application Programming Interface) using Python.
- **Snapcraft (Snap packages):** A software packaging and deployment system developed by Canonical. Unlike traditional package managers, snaps bundle the application and all its dependencies into a single, sandboxed compressed file, ensuring cross-platform consistency and strict security confinement.
- **ONNX (Open Neural Network Exchange):** An open-source format built to represent machine learning models. It allows models trained in one framework (like PyTorch or TensorFlow) to be transferred and executed in different environments, such as the ARM64 architecture of the ctrlX controller.
- **SQLite:** A C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine. It does not require a separate server process, making it ideal for embedded industrial applications.
- **PCA (Principal Component Analysis):** A mathematical technique used for dimensionality reduction that transforms a large set of variables into a smaller one that still contains most of the information in the large set, by creating new uncorrelated variables called principal components.
- **SVD (Singular Value Decomposition):** A linear algebra technique used to break down a complex matrix into simpler, meaningful components. In machine learning, it is frequently used alongside PCA to reduce the number of features while preserving the dataset's core structure.
- **HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise):** A data clustering algorithm that finds groups of highly dense points in a dataset and identifies outliers as noise. It is particularly effective because it does not require the user to specify the number of clusters in advance.
- **HTTPX:** A fully featured HTTP client for Python 3. It is used as an alternative to the standard requests library in sandboxed environments because it manages SSL certificate verification independently of the host operating system's trust store.
- **Data Layer:** An internal communication interface on the ctrlX CORE that provides applications with real-time access to motor axis data, sensor values, and control commands.